

# Social Media Platform-Case Study (Assignment-Day3)

## CREATE & READ Operations:

```
PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017
responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server
-----

socialMediaRS [direct: primary] test> use socialMediaDB
switched to db socialMediaDB
socialMediaRS [direct: primary] socialMediaDB> show collections

socialMediaRS [direct: primary] socialMediaDB>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
socialMediaRS [direct: primary] socialMediaDB>
• PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts\insertUsers.js
Users Inserted Successfully
• PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts\insertPosts.js
Posts Inserted Successfully
• PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts\insertComments.js
Comments Inserted Successfully
• PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts\insertFollowers.js
Followers Inserted Successfully
• PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts\insertActivityLogs.js
Activity Logs Inserted Successfully
• PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts\checkData.js
===== USERS =====
===== POSTS =====
===== COMMENTS =====
===== FOLLOWERS =====
===== ACTIVITY LOGS =====
```

## UPDATE & DELETE Operations:

The screenshot shows a VS Code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project structure for 'mongodb-replication-socialmedia' with folders 'data' and 'scripts'. The 'scripts' folder contains files like 'checkData.js', 'deleteData.js', 'failoverDemo.js', 'insertActivityLogs.js', 'insertComments.js', 'insertFollowers.js', 'insertPosts.js', 'insertUsers.js', 'setupReplica.js', and 'updateData.js'. The terminal shows the execution of 'mongosh --port 27017 scripts/updateData.js' and 'mongosh --port 27017 scripts/deleteData.js'. The output shows that data was deleted successfully. The terminal also shows the output of 'show collections' and 'db.users.find().pretty()', displaying a list of users with their IDs, usernames, emails, bios, follower counts, and following counts. The output shows two users: 'akshita' and 'rahal'.

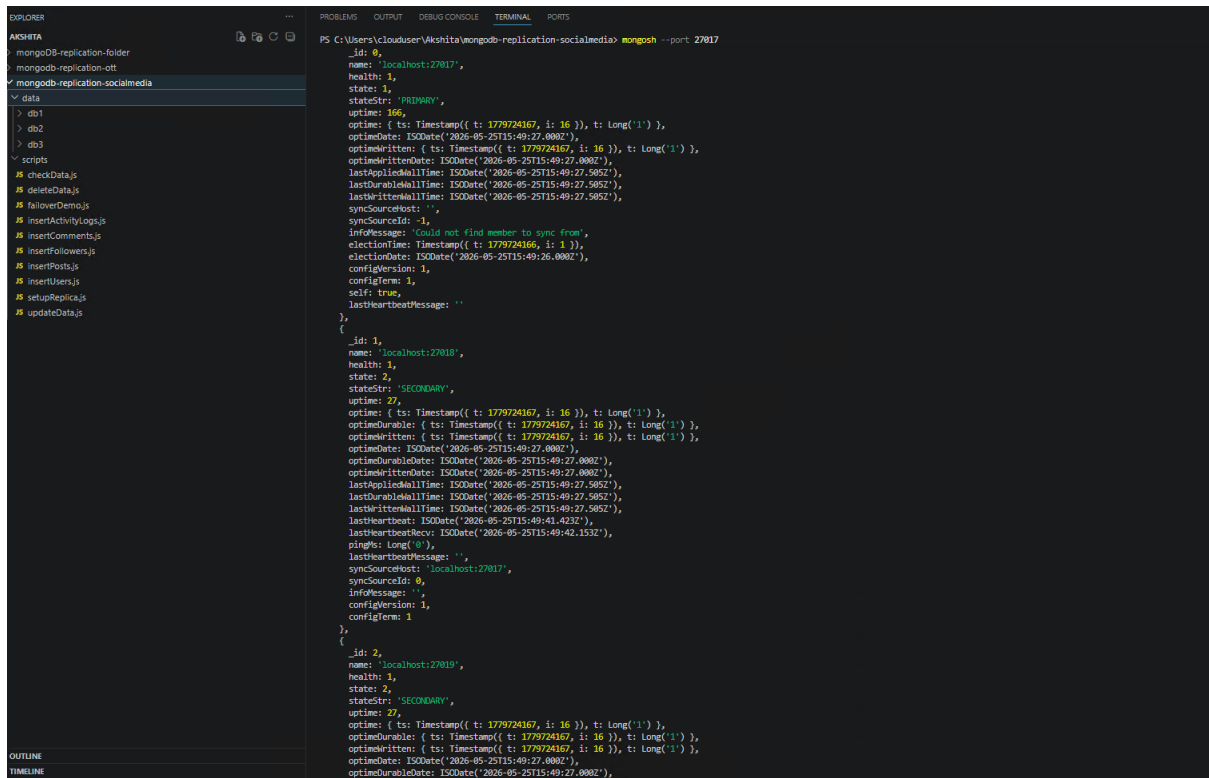
```
PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts/updateData.js
Data Deleted Successfully
PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017 scripts/deleteData.js
Data Deleted Successfully
PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017
Current Mongosh Log ID: 6a147424248c77c88dabc113
Connecting to: mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=
Using MongoDB: 8.3.2
Using Mongosh: 2.8.3
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-05-25T15:47:01.053+00:00: Access control is not enabled for the database. Read and write access to data
2026-05-25T15:47:01.054+00:00: This server is bound to localhost. Remote systems will be unable to connect t
responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server
-----

socialMediaRS [direct: primary] test> use socialMediaDB
switched to db socialMediaDB
socialMediaRS [direct: primary] socialMediaDB> show collections
activity_logs
comments
followers
posts
users
socialMediaRS [direct: primary] socialMediaDB> db.users.find().pretty()
[
  {
    _id: ObjectId('6a147282f1a0fe1d06abc114'),
    userId: 'U101',
    username: 'akshita',
    email: 'akshita@gmail.com',
    bio: 'Full Stack Developer',
    followersCount: 250,
    followingCount: 180
  },
  {
    _id: ObjectId('6a147282f1a0fe1d06abc115'),
    userId: 'U102',
    username: 'rahal',
    email: 'rahal@gmail.com',
    bio: 'Photographer',
    followersCount: 400,
    followingCount: 220
  }
]
socialMediaRS [direct: primary] socialMediaDB> db.posts.find().pretty()
[
  {
    _id: ObjectId('6a1472945c3e307532abc114'),
    postId: 'P101',
    userId: 'U101',
    caption: 'MongoDB Replication Learning',
    hashtags: [ 'mongodb', 'database' ],
    likesCount: 120,
    createdAt: ISODate('2026-05-25T16:02:28.766Z')
  },
  {
    _id: ObjectId('6a1472945c3e307532abc115'),
    postId: 'P102',
    userId: 'U102',
```

## Replica Set Operations

### setupReplica.js:

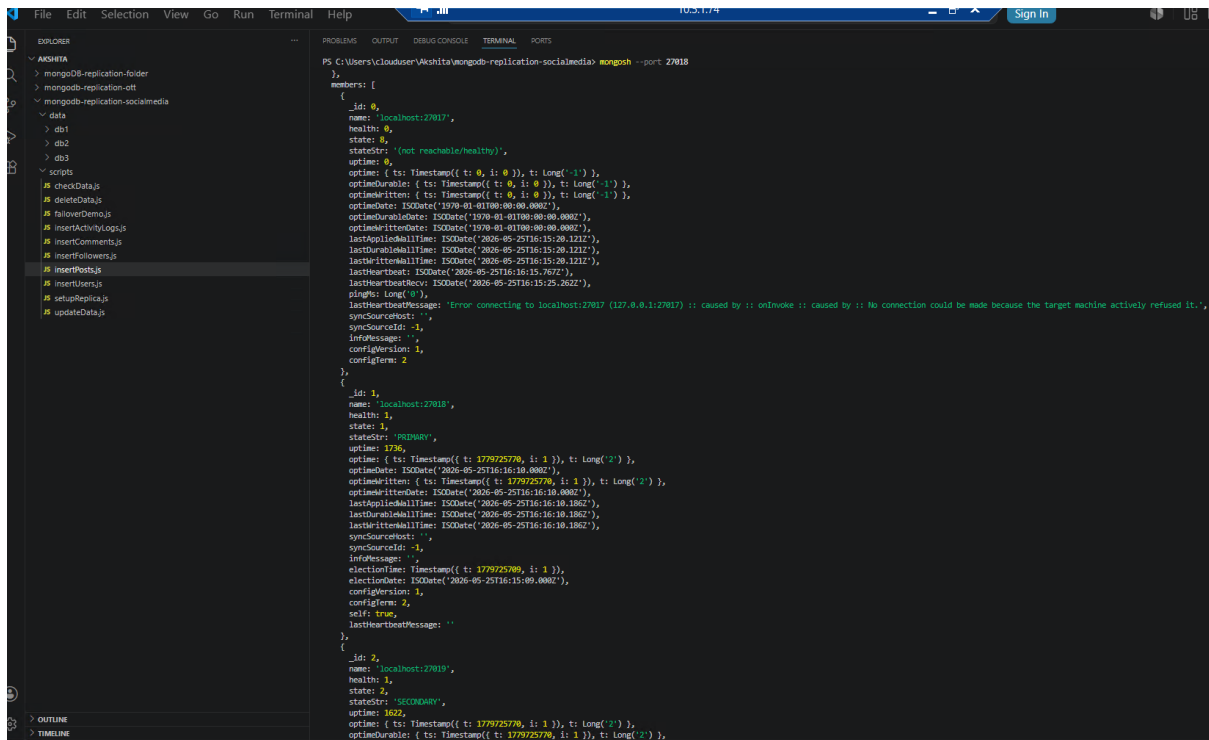


```
AKSHITA
└─ mongoDB-replication-folder
  └─ mongoDB-replication-ott
    └─ mongoDB-replication-socialmedia
      └─ data
        └─ db1
          └─ db2
            └─ db3
              └─ scripts
                └─ checkData.js
                └─ deleteData.js
                └─ failoverDemo.js
                └─ insertActivityLogs.js
                └─ insertComments.js
                └─ insertFollowers.js
                └─ insertPosts.js
                └─ insertUsers.js
                └─ setupReplica.js
                └─ updateData.js

OUTLINE
TIMELINE

PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27017
{
  _id: 0,
  name: 'localhost:27017',
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 166,
  optime: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeDate: ISODate('2026-05-25T15:49:27.000Z'),
  optimeWritten: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeWrittenDate: ISODate('2026-05-25T15:49:27.000Z'),
  lastAppliedWallTime: ISODate('2026-05-25T15:49:27.505Z'),
  lastDurableWallTime: ISODate('2026-05-25T15:49:27.505Z'),
  lastWriteWallTime: ISODate('2026-05-25T15:49:27.505Z'),
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: 'Could not find member to sync from',
  electionTime: Timestamp({ t: 1779724166, i: 1 }),
  electionDate: ISODate('2026-05-25T15:49:26.000Z'),
  configVersion: 1,
  configTerm: 1,
  self: true,
  lastHeartbeatMessage: ''
},
{
  _id: 1,
  name: 'localhost:27018',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 27,
  optime: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeDate: ISODate('2026-05-25T15:49:27.000Z'),
  optimeDurableDate: ISODate('2026-05-25T15:49:27.000Z'),
  optimeWrittenDate: ISODate('2026-05-25T15:49:27.000Z'),
  lastAppliedWallTime: ISODate('2026-05-25T15:49:27.505Z'),
  lastDurableWallTime: ISODate('2026-05-25T15:49:27.505Z'),
  lastWriteWallTime: ISODate('2026-05-25T15:49:27.505Z'),
  lastHeartbeat: ISODate('2026-05-25T15:49:41.423Z'),
  lastHeartbeatRecv: ISODate('2026-05-25T15:49:42.153Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 1,
  configTerm: 1
},
{
  _id: 2,
  name: 'localhost:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 27,
  optime: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1779724167, i: 16 }), t: Long('1') },
  optimeDate: ISODate('2026-05-25T15:49:27.000Z'),
  optimeDurableDate: ISODate('2026-05-25T15:49:27.000Z')
}
```

### failoverDemo.js:



```
File Edit Selection View Go Run Terminal Help
AKSHITA
└─ mongoDB-replication-folder
  └─ mongoDB-replication-ott
    └─ mongoDB-replication-socialmedia
      └─ data
        └─ db1
          └─ db2
            └─ db3
              └─ scripts
                └─ checkData.js
                └─ deleteData.js
                └─ failoverDemo.js
                └─ insertActivityLogs.js
                └─ insertComments.js
                └─ insertFollowers.js
                └─ insertPosts.js
                └─ insertUsers.js
                └─ setupReplica.js
                └─ updateData.js

OUTLINE
TIMELINE

PS C:\Users\clouduser\Akshita\mongodb-replication-socialmedia> mongosh --port 27018
{
  members: [
    {
      _id: 0,
      name: 'localhost:27017',
      health: 0,
      state: 9,
      stateStr: '(not reachable/healthy)',
      uptime: 0,
      optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('1') },
      optimeWritten: { ts: Timestamp({ t: 0, i: 0 }), t: Long('1') },
      optimeDate: ISODate('1970-01-01T00:00:00.000Z'),
      optimeDurableDate: ISODate('1970-01-01T00:00:00.000Z'),
      optimeWrittenDate: ISODate('1970-01-01T00:00:00.000Z'),
      lastAppliedWallTime: ISODate('2026-05-25T16:15:20.121Z'),
      lastDurableWallTime: ISODate('2026-05-25T16:15:20.121Z'),
      lastWriteWallTime: ISODate('2026-05-25T16:15:20.121Z'),
      lastHeartbeat: ISODate('2026-05-25T16:15:15.767Z'),
      lastHeartbeatRecv: ISODate('2026-05-25T16:15:25.202Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: 'Error connecting to localhost:27017 (127.0.0.1:27017) :: caused by :: onInvoke :: caused by :: No connection could be made because the target machine actively refused it.',
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      configVersion: 1,
      configTerm: 2
    },
    {
      _id: 1,
      name: 'localhost:27018',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 1736,
      optime: { ts: Timestamp({ t: 1779725770, i: 1 }), t: Long('2') },
      optimeDate: ISODate('2026-05-25T16:16:10.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1779725770, i: 1 }), t: Long('2') },
      optimeWrittenDate: ISODate('2026-05-25T16:16:10.000Z'),
      lastAppliedWallTime: ISODate('2026-05-25T16:16:10.186Z'),
      lastDurableWallTime: ISODate('2026-05-25T16:16:10.186Z'),
      lastWriteWallTime: ISODate('2026-05-25T16:16:10.186Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1779725769, i: 1 }),
      electionDate: ISODate('2026-05-25T16:15:09.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 2,
      name: 'localhost:27019',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1622,
      optime: { ts: Timestamp({ t: 1779725770, i: 1 }), t: Long('2') },
      optimeDurable: { ts: Timestamp({ t: 1779725770, i: 1 }), t: Long('2') },
      optimeWritten: { ts: Timestamp({ t: 1779725770, i: 1 }), t: Long('2') },
      optimeDate: ISODate('2026-05-25T16:16:10.000Z')
    }
  ]
}
```